

# **Fine-tuning, Reinforcement Learning and Parameter-Efficient Adaptation**

**Dimitris Tsirmpas**

Athens University of Economics and Business

Archimedes, Athena Research Center | June 18, 2026

# **What we will be talking about**

- 1. Naively Finetuning LLMs**
- 2. Making finetuning practical through LoRA**
- 3. Making finetuning scalable through QLoRA**

## Section 1

# **Naively Finetuning LLMs**

# Why finetune LLMs?

- **Prompting:** Changes **input behavior**. We guide the model with instructions and examples.
- **Fine-Tuning:** Changes **model parameters**. We switch the **knowledge** of the model.

## Reminder

**Fine-Tuning (FT):** The process of further training a pre-trained model on a smaller, domain-specific dataset to adapt its general knowledge to a specialized task.

## Example: Domain-specific tasks

### Part-of-Speech (POS) Tagging

The process of assigning grammatical categories to each word in a sentence.

#### POS Tagging

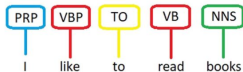


Figure: <https://byteiota.com/pos-tagging/>

## Example: Domain-specific tasks

### Biomedical POS Tagging

- Patterns absent or rare in general-domain text.
- Prompting probably not enough due to **missing knowledge**.

| Word       | POS Tag |
|------------|---------|
| Homozygous | JJ      |
| null       | JJ      |
| mutants    | NNS     |
| die        | VBP     |
| shortly    | IN      |
| after      | RB      |
| birth      | NN      |

Table: Example from the CRAFT dataset.

# Why not finetune LLMs?

- **High Resource Cost:**
  - **Computational Power:** Full FT requires massive hardware requirements and thousands of compute hours for large models (e.g., Llama-70B).
  - **Storage:** Each specialized task requires a separate, full copy of the massive foundational model.

# Why not finetune LLMs?

- **Operational Overhead:**

- Training is **slow**.
- Deployment becomes complex, managing **multiple large models** for different applications.
  - A single 13B model may require 26GB of storage.
  - 10 finetuned models may thus require 260GB.
- High risk of **Catastrophic Forgetting**—the model forgets its general knowledge while learning the specific task.



# Why not finetune LLMs?

## BERT

- **Model Size:** ~340M Parameters
- **VRAM Requirement:**  $\approx$  8-12GB
- **Training Time:** Minutes to a few hours

## Llama 70B

- **Model Size:** ~70 Billion Parameters
- **VRAM Requirement:**  $\approx$  1000GB+
- **Training Time:** Days to Weeks (even with specialized hardware)

LLMs are **too large** to run full-finetuning. But do we need to?

## Section 2

# **Making finetuning practical through LoRA**

# Parameter-Efficient Fine-Tuning (PEFT)

- Instead of finetuning the whole model, why not finetune **only a small part**?
  - Similar in concept to finetuning a classification head.
  - Introduce and train a tiny number of new, small matrices.
- **Goal:** Achieve performance benefits of Fine-Tuning while **drastically reducing the computational burden**.

# PEFT: LoRA

- Low-Rank (LoRA) Matrix Decomposition is a specific PEFT algorithm that exploits the fact that weight updates in LLMs have a “low intrinsic rank”.
- Low rank matrices enable us to scalably represent information.

# Low-rank matrices

What is a matrix?

A matrix is essentially a organized collection of data (rows and columns). For large datasets, these matrices can become incredibly complex, requiring massive storage and computational power.

Example: Data matrix

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 1 & 2 & 3 & 4 & 5 \\ 3 & 6 & 9 & 12 & 15 \\ 4 & 8 & 12 & 16 & 20 \end{bmatrix}$$

# Low-rank matrices

## Matrix rank

- The 'true complexity' or the number of genuinely independent patterns within the matrix  $M$ .
- Intuitively, it tells us how many underlying 'ingredients' or fundamental directions are needed to describe the entire matrix.

## Example: Cooking

Think of a restaurant with 100 dishes but only 3 main food styles: Italian, Mexican, and Vegetarian. Each dish can be described as a mix of these few styles.

# Low-rank matrices

## Rank Factorization

If a matrix  $M$  has a low rank  $k$ , we can approximate it (or exactly represent it) by factoring it into a product of two simpler matrices.

Example: Rank 1 matrix

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 1 & 2 & 3 & 4 & 5 \\ 3 & 6 & 9 & 12 & 15 \\ 4 & 8 & 12 & 16 & 20 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 3 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \end{bmatrix}.$$

# LoRA: Low-rank decomposition

$$\underbrace{A_{m \times n}}_{\text{large matrix}} \approx \underbrace{U_{m \times r}}_{\text{left factors}} \underbrace{V_{r \times n}}_{\text{right factors}}, \quad r \ll \min(m, n).$$

$$A_{10000 \times 5000} \approx U_{10000 \times 50} V_{50 \times 5000},$$

$$\underbrace{10000 \times 5000}_{50,000,000 \text{ parameters}} \longrightarrow \underbrace{10000 \times 50 + 50 \times 5000}_{750,000 \text{ parameters}}.$$



# LoRA: Mathematical Foundation

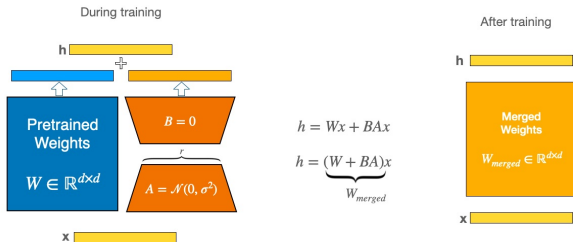


Figure: [https://huggingface.co/docs/peft/main/conceptual\\_guides/lora](https://huggingface.co/docs/peft/main/conceptual_guides/lora)

- $W \in \mathbb{R}^{d \times k}$  weights.
- $x$  input,  $h$  output.
- We add  $BAx$  to the output.
- $B \in \mathbb{R}^{d \times r} \rightarrow$  zeros.
- $A \in \mathbb{R}^{r \times k} \rightarrow$  Gaussian.
- $r \ll \min(d, k)$ , hence  $A, B$  are much smaller than  $W$ .<sup>17/28</sup>

# LoRA: Algorithm

▷ *====Initialization====* ◁

**for all**  $W \in all\_weights$  **do**

└ Set  $W$  to read-only ▷ *Original knowledge preserved*

**for all**  $W_u \in target\_modules$  **do**

└ Initialize  $A$  randomly

└ Initialize  $B = 0$  ▷ *Contributes zero to the output initially*

▷ *====Backpropagation====* ◁

**for all**  $X \in input\_embeds$  **do**

└  $\Delta W = \frac{\alpha}{r} AB$

└  $H = WX + \Delta WX$  ▷ *Output = original + adaptation*

▷ *====End of training====* ◁

$W' = W + \frac{\alpha}{r}(AB)$  ▷ *Merge original and adaptation*

# LoRA: Practical Considerations

- How many ranks do I use?
  - $r \in [1, 16]$ : fast, easy, recommended for minor adjustments.
  - $r \in [16, 64]$ : recommended for most domain adaptation scenarios.
  - $r \in [64, 256]$ : may hit computational bottlenecks, only use for dramatic shifts in domain and knowledge.
- How do I set  $\alpha$ ?
  - Usually set to a multiple of  $r$ .
  - Very low  $\alpha \rightarrow$  minimal contribution. Very high  $\alpha \rightarrow$  destruction of model knowledge.
  - $\alpha = 2r$  usually used as a starting point.

# LoRA: Practical Considerations

- What modules do I change?
- Start from the top of this list and go down as required:
  1. **Query (q\_proj) and Value (v\_proj):** Most frequently adapted; control **how the model attends** to input.
  2. **Key (k\_proj):** Increases **information retrieval** capacity.
  3. **Output (o\_proj):** Less common, but useful for modifying **integrated** attended information.
  4. **Feed-forward Networks (MLP):** Possible to adapt, but generally offer less parameter benefit compared to attention layers.

# LoRA: Practical Considerations

- Should we merge the LoRA weights?

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}(\mathbf{AB})$$

## Yes

- Lowers GPU memory and execution time.
  - More significant if  $r$  is large.
- Deployment may be easier.

## No

- You can use the same model with many finetuned adapters which are dynamically selected depending on your task.

## Section 3

# **Making finetuning scalable through QLoRA**

# QLoRA: Quantization

- Quantization refers to dropping the size (not number) of the LLM parameters.
- Normally, each parameter is stored in a 16 bit floating point.
- Quantization can drop them to usually 4 bits, resulting in  $\times 4$  less GPU memory and inference costs.



Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>

# QLoRA: Quantization

- Quantization is one of the **most useful tools** at your disposal.
- Allows for training and inference in consumer-grade hardware.
- We will return to this in Module 5.

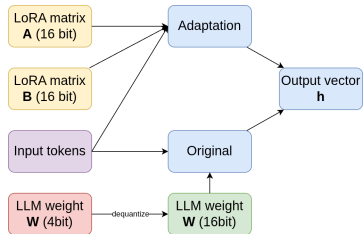


Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>



# QLoRA: Concept

- Load a 4-bit quantized model using a normal approximation of weights.
- Run LoRA in full precision, **dequantizing** the model weights **one at a time**.
- Continue as normal.



# QLoRA: Algorithm

▷ *====Initialization=====*

◁

▷ *Same as before*

◁

▷ *====Backpropagation=====*

◁

**for all**  $X \in \text{input\_embeds}$  **do**

$$\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{A} \mathbf{B}$$

$\mathbf{H} = \text{DEQUANT}(\mathbf{W}) X + \Delta \mathbf{W} X$  ▷ *Output = original + adaptation*

▷ *====End of training=====*

◁

▷ *Only attempt dynamic merge. Ignore this advice at your own peril, or if you really know what you are doing.*

◁

# QLoRA: Pros and cons

## Pros

- Frozen weights no longer monopolize GPU memory and inference time.
- Allows training in consumer-grade hardware.
- Massively reduces training times.

## Cons

- End-result is a quantized → less powerful model.
- Dequantization imposes a  $\approx 10\%$  tax on training times.
- Merging is now much messier.

# Summing up

- Full finetuning for LLMs is out of the question.
- However, weight updates are usually **low-rank**.
- Thus, we can **approximate** full finetuning with LoRA.
- We may use QLoRA instead to make it much more **scalable**.